

Archiving Years of Dataverse Audit History Before You Prune It — A Pragmatic, Open-Source Pattern

A note up-front. This post documents an architectural choice — not a Microsoft product, not a Microsoft offering, and not officially supported. For most Dataverse-to-Azure replication needs the first-class options are [Azure Synapse Link for Dataverse](#) and [Microsoft Fabric Link for Dataverse](#); if either fits your scenario, prefer them.

The specific gap this post addresses is the **audit table**. Synapse Link / Fabric Link can replicate audit *header* rows like any other table, but the per-change payload returned by `RetrieveAuditDetails` (old value vs new value, attribute mask, related-record diffs) is computed by Dataverse on demand and is not part of the table contents the link replicates. So if your need is "keep the row-level *what changed* evidence after I prune from Dataverse", you need to call `RetrieveAuditDetails` while the source row still exists — which is what this pattern does.

The audience this is written for: enterprises with years of accumulated audit history that, for whatever reason, *couldn't* adopt Synapse Link / Fabric Link — typically because their analytics platform of record sits outside Azure (most commonly Snowflake), or because internal review hasn't approved the link in their tenant — and who need a defensible cold archive *before* Dataverse's audit-deletion job runs. Treat this as a reference implementation you can adapt, fork, or discard.

Why the audit table is special

Most of what Dataverse exports through Synapse Link is *current state*. The **audit table** is different in two ways:

1. It's an immutable, append-only record of *who changed what, when, and from where* — the closest thing Dataverse has to a forensic ledger.
2. The valuable part of an audit row is not the row itself; it's the diff (old value → new value, attribute mask, related-record context) returned by the bound `RetrieveAuditDetails` function. That diff is computed at read-time against the still-living source row. Once Dataverse deletes the audit, no API call can reconstruct it.

That combination makes the audit table the single most useful Dataverse table for:

- Regulatory and compliance investigations
- "Why did this opportunity status change in Q3 of 2022?" forensic queries (years after the fact)
- Internal analytics on user behaviour and process adoption

It's also the table that grows the fastest. The Dataverse retention default is 90 days, but in practice many enterprises change that — they set it to several years, or to *never delete* — because nobody wants to be the admin who threw away evidence. The result, often after a quiet five or seven years, is an audit table sitting on tens of GB to multiple TB of capacity, dominating the entitlement bill, and never touched in normal operations.

At that point the storage conversation becomes unavoidable. The realistic choices are:

- 1. **Keep buying entitlement.** Predictable, but unbounded.
- 2. **Move the cold tail somewhere cheaper that you control, then let Dataverse's audit-deletion job reclaim the space.** The hot months stay in Dataverse where users expect them; the years of historical evidence live in your own storage account, queryable when you need them.

This pattern is for option 2 — specifically, for the *one-time bulk export of multi-year history*, with the option to keep a slow trickle running afterwards if you want to top up.

A crucial point that often gets lost: this pipeline does not need to run live. It is perfectly reasonable to be deliberately months or years behind real-time. The goal is to get a defensible copy of *cold* data out — the rows you are about to allow Dataverse to delete — not to mirror the audit feed in real time.

What "good" looks like for an external audit copy

Before showing any code, here's the rubric I held this design to. If you build your own, hold yours to the same rubric:

Property	Why it matters
Idempotent	Re-running the same time window must not duplicate rows. Network blips happen.
Crash-safe (exactly-once-effective)	If the process dies mid-window, the next run must replay the same window cleanly. The watermark advances <i>only</i> after the data is durable in the destination.
Bounded memory	A backlog of millions of audits cannot be loaded all at once.
Backpressure-aware	Dataverse rate-limits aggressively. Throttle responses must not drop rows.
Observable	Every window logs <code>[entity] mode=BACKLOG/LIVE, lag=Nmin, window=10min, records=N</code> so you can watch it work.
Sink-agnostic	The "where does it land" decision is config, not code. Storage choices change; the orchestrator shouldn't.
Field-level discretion	Audit details can carry PII. The pattern should let admins narrow which attributes leave the platform.

The pattern

The pipeline is conceptually four stages, repeated per entity, per time window:

```
Per entity (account, contact, ...):  
  1. Read watermark (lastSyncEnd) from sink's state container  
  2. Compute next time window: [lastSyncEnd, lastSyncEnd + N min)  
  3. Fetch audit headers from Dataverse Web API:  
     $filter=createdon ge <start> and createdon lt <end>  
  4. For each header → call RetrieveAuditDetails (bound function)  
  5. (Optional) Project to allow-listed attributes only  
  6. Upsert each audit doc into sink (id = auditid, idempotent)  
  7. Advance watermark ONLY if step 6 fully succeeded  
  
Loop until caught up; then either exit (function) or sleep (daemon)
```

Three details in this picture do most of the resiliency work. They are deceptively simple:

Detail 1: Half-open time windows (ge / lt)

```
[09:00:00, 09:10:00) → window 1  
[09:10:00, 09:20:00) → window 2
```

The boundary moment (09:10:00.000) belongs to window 2, not window 1. So adjacent windows never overlap and never gap, no matter how many times you replay. This is the same trick Kafka uses for offsets — it's why you can run the loop with confidence.

Detail 2: The destination document key is the Dataverse auditid GUID

Dataverse already assigns a globally unique GUID to every audit row. That GUID becomes the document `id` in the sink. So when you upsert the same audit twice, the second write is a no-op overwrite of the first — idempotency for free, no client-side dedupe table to maintain.

Detail 3: The watermark moves *after* the write, not with it

The naive version of this pipeline does:

```
records = fetch_window(...)  
sink.write(records)           # 950/1000 succeed; 50 fail with 429  
sink.update_watermark(window_end) # ← BUG: window now "done", 50 rows lost forever
```

The resilient version raises a typed exception when *any* record fails, and the watermark update is conditional on a clean write:

```
try:
    written = sink.write(records)
    sink.update_watermark(window_end)
except SinkPartialWriteError:
    # Watermark stays put. Next loop iteration replays the same window.
    # Rows that already landed become no-op upserts. Rows that failed get retried.
    raise
```

This single change is the difference between "best effort" and "exactly-once-effective." It's also the mistake most often made when people roll their own.

Going deeper. The exact API call shape, why concurrency is bounded globally instead of per-entity, the state-container schema, the half-open math, the worked test runs (smoke / partial-failure drill / full-backlog harness), and the step-by-step "how to add a sink for your own destination" guide all live in the repo's design notes:

[unified-deployment/DESIGN.md](#)

The intent is to keep the blog itself focused on the *shape of the problem and the choice of pattern*; the depth that matters when you actually fork the code lives next to the code.

Choosing where it lands

The orchestrator is sink-agnostic — it talks to a single `AuditSink` interface (`get_state` , `update_state` , `write_audits`) and the destination is a config switch, not a code change. The reference implementation ships with four production-shaped sinks plus a no-op for testing. None of them is *the* answer; they map to platforms enterprises already operate:

Sink	When to consider it
Azure Cosmos DB (NoSQL API)	Operational lookups — "show me everything user X did to record Y in 2022" in milliseconds. Hierarchical partition keys (<code>/entity</code> + <code>/auditYearMonth</code>) keep partitions small as the archive grows over years. Document TTL doubles as a retention policy if you want one. Serverless mode suits a slow-trickle archive workload.
Azure Data Lake Storage Gen2 (Parquet)	The cheap-cold-storage option. Years of audit history land as partitioned Parquet files (<code>entity=.../year=.../month=.../</code>), readable from Fabric notebooks, Synapse Serverless SQL, Databricks, or any Parquet engine. Costs scale with bytes, not throughput — ideal when the archive is rarely queried but must exist.
OneLake (Parquet)	Same Parquet shape as ADLS, but landed inside a Microsoft Fabric Lakehouse. Immediately queryable from a Fabric SQL endpoint, notebooks, and Power BI without further plumbing. The natural choice if your downstream BI is Fabric.
Snowflake (MERGE INTO)	The natural choice when Snowflake is already the analytics platform of record and adding a separate Microsoft analytics estate just for audit data isn't on the table. <code>MERGE INTO ... ON audit_id</code> keeps the same idempotency contract as the Cosmos upsert, and the warehouse stays paused between archival batches.
No-op (logs only)	First-day connectivity testing. Confirms the Dataverse side works before you provision any storage.

A reasonable default split many enterprises arrive at:

- **ADLS Gen2 / OneLake** (or **Snowflake**, if that's your platform) holds the durable historical archive — cheap, partitioned, queryable when (rarely) needed.
- **Cosmos DB** holds the most recent N months for fast operational lookup if there is a use case for it; otherwise skip it entirely.

Adding a sink for storage you already own (e.g., BigQuery, Redshift, on-prem object storage) is roughly 100 lines of Python and one factory entry.

What real-world numbers will look like

The worked test run above shows what a clean sandbox run looks like. Real numbers in your tenant will vary by orders of magnitude depending on:

- How many entities have auditing enabled
- The shape of `RetrieveAuditDetails` calls (more changed attributes per row = more bytes per call)
- Dataverse Web API rate limits applicable to your environment
- Concurrency you allow (`max_concurrent_entities` in the config)
- Sink throughput (Cosmos serverless RU autoscale, ADLS upload bandwidth, Snowflake warehouse size)

The useful operational signal is not the absolute throughput — it's that the throughput is *stable* and the per-window log lines tick predictably. If they don't, look at lag, sink errors, or 429 responses from Dataverse before scaling up concurrency.

Things to be honest about

Because this is a "build it yourself" pattern, here are the trade-offs you take on. These are not flaws — they are the price of doing the export yourself instead of standing up a managed pipeline, and you should price them in:

1. **You own the watermark.** If the destination is wiped without the state container, the next run will start over from whatever seed timestamp you give it (usually the earliest `createdOn` in the audit table for first runs, or your last known `lastSyncEnd` for resumes). Treat the state container as production data — back it up.
 2. **Schema is your problem.** When Dataverse adds a new field to the `audit` table, the change applies in production. Your sink will keep working — the field just shows up in the JSON blob — but if you're projecting columns (e.g., the attribute allow-list), update the config.
 3. **Permissions are your problem.** The application user needs read on `audit` (and `RetrieveAuditDetails` privileges per entity). The Power Platform admin centre handles this, but it's an out-of-band step.
 4. **Cost shape changes.** You're trading "Dataverse storage entitlement" for "ADLS bytes," "Cosmos RUs," or "Snowflake credits." Run the math on your record volume before committing — partitioned Parquet on cool/cold object storage is usually the cheapest by an order of magnitude when the archive is read rarely.
 5. **Auditing your auditor.** If this pipeline becomes evidence in a compliance investigation, the *pipeline itself* needs an audit trail. The reference implementation stamps every output document with a `runId` (UUID per window) and a `processedAt` timestamp. Keep the orchestrator logs.
 6. **It is not a Microsoft product.** I'll repeat this because it matters: if you adopt it, you own the operations, the upgrades, and the on-call pager.
-

When to reach for it (and when not to)

Reasonable fit:

- You have multiple years of accumulated audit history in Dataverse and need to move the cold tail off the platform before pruning to reclaim entitlement.
- The evidence you actually care about is the `RetrieveAuditDetails` payload (old value → new value), not just the audit header row — so a table-level replicator alone wouldn't preserve what you need.

- You're comfortable running this as a *batch job* — once for the historical backfill, then perhaps quarterly or annually to top up — rather than as a live continuous feed. Being deliberately months or years behind real-time is fine and often desirable.
- You want a portable, scriptable export that targets storage you already own — your Azure subscription (ADLS Gen2, OneLake, Cosmos DB), your Snowflake account, or another platform you can write a thin sink for.
- You want field-level control over which attributes leave the platform — useful when audit details contain regulated data.

Not a good fit:

- You only need *current state* of business tables (account, contact, opportunity). Use Synapse Link / Fabric Link — they do exactly that and you don't need this pattern.
- You only need audit *header* rows (who/when/which table) and not the per-attribute change detail. Synapse Link / Fabric Link can replicate the audit table itself; consider that first.
- You need sub-second freshness in the destination. The pattern's natural cadence is one window length (10 min in the reference config); for true real-time, use Dataverse webhooks or change-tracking APIs.
- You don't have somewhere to operate a small Python container, function, or scheduled job — even an annual one.
- You don't have an internal owner who can be paged when the schedule fails.

The reference implementation

The code that backs this post lives at <https://github.com/SweetsNSavories/DataverseAuditLogSyn> under MIT, with no warranty. The `unified-deployment` folder is the version this post describes — single Python codebase, swap sinks via `config.json`, runs locally / in a container / as an Azure Function.

If you want the implementation depth this post deliberately leaves out — exact API shapes, watermark math, partial-failure drill, sink-author checklist, hosting variants, observability hooks — it all lives in one place:

- [unified-deployment/DESIGN.md](#)

If something feels wrong for your environment, change it — that's the whole point of publishing it.

Closing

For most Dataverse-to-Azure replication needs, Synapse Link / Fabric Link is the right starting point and you should evaluate them first. The narrower problem this pattern solves is the audit table specifically — where the evidence you usually care about (the `RetrieveAuditDetails` diff) lives outside what a table-

level replicator carries, and where the most common business trigger is *"we have years of audit data, we've never deleted any of it, and the storage bill is now a problem."*

If that's where you're sitting — multi-year backlog, Synapse Link / Fabric Link not on the table for whatever reason (often because your analytics platform of record is Snowflake or otherwise outside Azure), looking for a defensible way to move the cold tail to storage you already own *before* you let Dataverse prune anything — this is one option among several. Adopt it, adapt it, or use it as the "here's the shape of the problem" sketch when you talk to your platform team about doing something more substantial.

Comments, forks, and "this is wrong because..." PRs welcome.